



← Back to student guides

 Docker

Containers

3 levels

133 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.



Part one of three. Three mistakes produce most of the errors below: you forget a container is one process, you forget the writable layer is temporary, or you confuse build time with run time. Start with the table, then the habits and practice cards.





Symptom	Real cause	Fix
Container exits instantly	The main process finished	A container lives as long as PID 1. Use <code>-it</code> , or run a real server
<code>docker run ubuntu</code> does nothing	Same cause: a shell with no terminal	<code>docker run -it ubuntu bash</code>
Cannot connect to the Docker daemon	The daemon is not running	Start Docker Desktop, or <code>sudo systemctl start docker</code>
permission denied ... <code>docker.sock</code>	Your user is not in the <code>docker</code> group	Add it, then log out and back in
Published port refuses connections	App listens on <code>127.0.0.1</code>	Bind <code>0.0.0.0</code> inside the container
Port mapping seems reversed	<code>-p</code> is host:container	<code>-p 8080:80</code> = localhost 8080 → container 80
<code>bind: address already in use</code>	Another container already has that host port	<code>docker ps</code> , then remove or use another port
<code>EXPOSE</code> did not expose anything	It is documentation only	Add <code>-p</code> to <code>docker run</code>
Build hangs forever	App start command placed in <code>RUN</code>	Start commands belong in <code>CMD</code> / <code>ENTRYPOINT</code>
<code>CMD</code> with single quotes fails	The exec form is JSON	Use double quotes: <code>CMD ["sh", "-c", "..."]</code>
Every build reinstalls dependencies	<code>COPY . .</code> before the install	Copy the manifest first, install, then copy source
Build sends hundreds of MB	No <code>.dockerignore</code>	Write one before your first build



A file is missing inside the image	Excluded by <code>.dockerignore</code> , or wrong <code>WORKDIR</code>	<code>docker run --rm img ls -la /app</code>
Data disappeared after <code>docker rm</code>	It was in the writable layer	Use a named volume
<code>bash: not found</code> on exec	Slim and Alpine images have no bash	<code>docker exec -it name sh</code>
Code changes not reflected	You rebuilt but ran the old tag	Rebuild and re-run, or bind-mount in dev
<code>docker logs</code> is empty	App writes to a file, not stdout	Log to stdout and stderr
Python logs appear late or not at all	stdout is buffered when not a terminal	<code>ENV PYTHONUNBUFFERED=1</code>
Disk full	Dangling images, stopped containers, build cache	<code>docker system df</code> , then prune
Compose app cannot reach the DB	Using <code>localhost</code> instead of the service name	Connect to <code>db:5432</code>
Cannot reach a sibling by name	On the default bridge, which has no DNS	Create a network, or use Compose
App starts before the DB is ready	<code>depends_on</code> waits for start, not readiness	Health check + <code>condition: service_healthy</code> , or retry in the app
"Permission denied" on a bind mount	Host and container UIDs differ	Match the UID, or use a named volume
<code>docker exec</code> says no such container	It already exited	<code>docker run -it --entrypoint sh img</code> instead
Secret still in the image after deleting it	Layers are additive	Rebuild without it, and rotate the credential



Write `.dockerignore` first

Before your first build. It keeps `.git`, `node_modules`, and `.env` out of the image and off the wire.



Dependencies before source

Copy the manifest, install, then copy the code. A one-character edit then rebuilds in seconds.



Never deploy `:latest`

Tag with a version or a commit SHA so you can always answer "what is running?"



Use `--rm` for throwaways

Each run without it leaves a stopped container behind. `--rm` keeps `docker ps -a` readable.



Add a non-root USER

Two lines. Containers default to root, and root in the container is root on the host kernel.



Log to stdout

It is what `docker logs`, log shippers, and every orchestrator expect.



Run it in the foreground first

Drop the `-d` when something is new, so the failure lands in your terminal instead of the logs.



Prefer Compose, even for one service

The file documents the ports, environment, and volumes that would otherwise live in your shell history.



Short, self-contained exercises. Each one takes a few minutes and ends in a result you can check on screen.

01 Prove a container is just a process

Run `docker run --rm -it alpine sh`. Inside: `ps aux`, `hostname`, `uname -r`. On your host, find that shell in `ps aux` and compare `uname -r`. Same kernel, two views.

02 Kill your own data on purpose

Run Postgres with no volume, create a table, `docker rm -f` it, start it again. Then repeat with `-v pgdata:/var/lib/postgresql/data`. The first round loses the table, the second keeps it.

03 Break the port mapping

Start your app bound to `127.0.0.1` with `-p 8000:8000`. Watch it refuse connections, then confirm the cause from inside with `ss -tlnp`. Fix it with `0.0.0.0`.

04 Time both COPY orders

Build, edit one character of source, rebuild, and note the time. Move `COPY.` above the install and repeat. Watch where `CACHED` stops in the build output.

05 Find a "deleted" secret

`RUN echo "SECRET=abc" > /tmp/k` then `RUN rm /tmp/k`. Rebuild, then `docker history --no-trunc img | grep SECRET`. The string is still in the layer that wrote it.

06 Cause three exit codes

Produce a 127 (a `CMD` binary that does not exist), a 1 (an exception on startup), and a 137 (`-m 16m`). Diagnose each using the debugging order rather than memory.

07 Prove DNS needs your own network

Start two containers with no network flag and try `ping other`. Then create a network, restart both on it, and try again.



`/etc/test` inside and read the refusal.

Debugging a container that will not start

01 Read the logs

`docker logs NAME` . Even a container that exited keeps its output until you remove it, so this works after a crash. You can read most failures straight out of that output.

02 Check the exit code

`docker ps -a` shows it.

0

= finished normally ·

1

= application error ·

125

= bad `docker run` flags ·

126

= command not executable ·

127

= command not found ·

137

= killed, usually out of memory ·

143

= stopped by SIGTERM.

03 Override the entrypoint to get a shell

`docker run -it --entrypoint sh myapp` starts the image without running your app, so you can look around and try the command by hand.



wrong `WORKDIR` or a file your `.dockerignore` excluded.

05

Run the command manually

Type the exact `CMD` yourself. The interactive error message carries more detail than the one in the logs.

06

Inspect the resolved config

`docker inspect NAME` shows the real entrypoint, command, env, mounts, and networks, after every layer of defaults and overrides. This settles "but I set that".

BASH

Copy

```
# 127 = command not found: check the binary exists and PATH is right
docker run -it --entrypoint sh myapp
> which uvicorn
> ls -la /app
> echo "$PATH"

# 137 = OOM-killed: give it more memory or find the leak
docker inspect NAME --format '{{.State.ExitCode}} {{.State.OOMKilled}}'
```

Drop the `-d` when something is new

Run it in the foreground and the output arrives in your terminal as it happens, instead of after a detached start and a trip to the logs. Add `-d` once it works.

Build-time versus run-time

Two things go wrong here: you put configuration in the image, or you expect a run-time flag to have changed the build.



- Happens once, during `docker build`
- Results become layers
- Install dependencies here

- Happens on `docker run`
- Nothing is saved to the image
- Configuration and secrets belong here

Never bake a secret in with `ENV` or `ARG`

Both end up in image metadata and in `docker history`, readable by anyone who can pull. Deleting the file in a later layer does not help, because the earlier layer still contains it. Pass secrets at run time with `-e`, `--env-file`, or a mounted file.

Making builds fast

Dockerfile

Copy

```
FROM python:3.11-slim

WORKDIR /app

# 1. Dependency manifest only: this layer is cached until it changes
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 2. Source last: edits only invalidate from here down
COPY . .

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Trick	Why
Manifest before source	The expensive install layer stays cached
<code>--no-cache-dir</code> on pip	The wheel cache is dead weight in the image



<code>npm ci</code> not <code>npm install</code>	Reproducible, and faster
One <code>RUN</code> for <code>apt-get update && install && rm -rf /var/lib/apt/lists/*</code>	Separate <code>RUN</code> s cache a stale index and ship the package lists
A good <code>.dockerignore</code>	Less context to send, fewer accidental cache busts
Watch for <code>CACHED</code> in the output	The last <code>CACHED</code> line marks where invalidation starts

DOCKERFILE

Copy

```
# Wrong: the stale index gets cached, and the lists stay in the image
RUN apt-get update
RUN apt-get install -y curl

# Right: one layer, cleaned up in the same layer
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
```

Choosing a base image

Base	Size	Use it when
<code>python:3.11</code>	~1 GB	You need build tools; fine for local experiments
<code>python:3.11-slim</code>	~150 MB	The sensible default for most apps
<code>python:3.11-alpine</code>	~50 MB	Size matters and you have tested it: musl breaks some wheels
<code>ubuntu:22.04</code>	~78 MB	You need a general-purpose OS and will install everything



native modules. You then compile from source, and the build ends up slower and larger than slim. Pick Alpine for a reason, not by default.

Always pin a tag. `FROM python:3.11-slim` is fine; `FROM python:latest` means your build changes under you without a single commit.

Develop inside the container

You do not need to rebuild the image after every code change. Bind-mount your source and let the app reload itself.

compose.override.yaml

Copy

```
services:
  api:
    build: .
    volumes:
      - ./app                # your source, live
      - /app/node_modules    # anonymous volume shields the image's deps
    environment:
      LOG_LEVEL: debug
    command: ['npm', 'run', 'dev']
```

BASH

Copy

```
docker compose up          # base + override, applied automatically
```

The second volume line is the one to get right. Mounting `./app` hides everything the image installed at `/app`, including `node_modules`. Adding an anonymous volume at `/app/node_modules` lets the image's installed dependencies show through the mount.



the Dockerfile or the dependency manifest changes. If you are rebuilding after every edit, your source is not mounted.

Inspect anything with `--format`

`docker inspect` returns a wall of JSON. A Go template turns it into the one answer you wanted. Keep these in your shell history.

BASH

Copy

```
# Why did it stop?
docker inspect api --format '{{.State.ExitCode}} {{.State.OOMKilled}} {{.State.Error}}'

# Which image is this container running?
docker inspect api --format '{{.Config.Image}} → {{.Image}}'

# What environment did it get?
docker inspect api --format '{{range .Config.Env}}{{println .}}{{end}}'

# What is mounted where?
docker inspect api --format '{{range .Mounts}}{{.Type}} {{.Source}} → {{.Destination}}{'

# Which networks and IPs?
docker inspect api --format '{{range $k, $v := .NetworkSettings.Networks}}{{$k}} {{$v.IP'

# Health check history
docker inspect api --format '{{json .State.Health}}'
```

Useful listing variants too:

BASH

Copy

```
docker ps -a --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
```



Platform gotchas

Docker behaves differently depending on where you run it. Check your row below before you assume the problem is in your code.

Platform	Watch out for
macOS / Windows	Docker runs in a Linux VM, so bind-mount file I/O is slower than native by a wide margin. Keep <code>node_modules</code> and build caches in volumes, not bind mounts
Apple Silicon	Builds produce <code>arm64</code> images. Deploying to an <code>amd64</code> server gives "exec format error". Build with <code>--platform linux/amd64</code>
Windows / WSL2	Keep your project inside the WSL filesystem (<code>~/code</code>), not <code>/mnt/c/...</code> . Cross-filesystem mounts slow every file read
Linux	<code>-v "\$(pwd)":/app</code> keeps host ownership, so UID mismatches cause permission errors that Mac and Windows users never see
All	<code>host.docker.internal</code> reaches the host from a container on Desktop; on plain Linux you need <code>--add-host=host.docker.internal:host-gateway</code>

"Works on my Mac, exec format error on the server"

An `arm64` image will not start on an `amd64` host, whatever the app inside it does.

Check the architecture before you debug anything else: `docker image inspect myapp --format '{{.Architecture}}'` .

Keeping your machine clean



```
docker system df          # what is using space, by category
docker system df -v       # per-image, per-container, per-volume detail
docker ps -a              # stopped containers still hold their layer
docker container prune    # remove all stopped containers
docker image prune        # dangling (untagged) images
docker builder prune      # build cache
docker system prune       # containers, networks, dangling images, cache
docker system prune -a --volumes # everything unused: read this twice first
```

Start with `docker system df` so you know which category is large. Most of the time it is the build cache or images, not containers.

--volumes deletes your data

`docker system prune -a --volumes` removes unused volumes, which includes the database of a project you stopped last week. Run `docker volume ls` and read the list first. Nothing else in Docker destroys work you cannot recover.

Small things to do from day one

Name your containers. `--name web` beats hunting for `nostalgic_bardeen` in `docker ps`.

Pin base image tags. Never `:latest` in a `FROM`, and never `:latest` in a deployment.

Set `ENV PYTHONUNBUFFERED=1` (or your language's equivalent) so `docker logs` shows output as it happens rather than in buffered bursts.

Use `--restart unless-stopped` for anything you want to survive a reboot.

Set `-m` and `--cpus` locally. A runaway container should hit a limit, not freeze your laptop.

Add `LABEL org.opencontainers.image.source` so anyone holding the image can find the repository that produced it.



```
docker run -d \  
  --name myapp \  
  -p 8000:8000 \  
  -e LOG_LEVEL=info \  
  -m 512m --cpus 1 \  
  --restart unless-stopped \  
  myapp:1.0
```

The equivalent, but readable and reviewable:

compose.yaml

Copy

```
services:  
  app:  
    build: .  
    ports: ['8000:8000']  
    environment:  
      LOG_LEVEL: info  
    mem_limit: 512m  
    cpus: 1  
    restart: unless-stopped
```

A starter setup to copy

Copy this into a new project and delete what you do not need. Every line here comes from somewhere on this page.

.dockerignore

Copy

```
.git  
.gitignore  
.github  
node_modules  
__pycache__
```



```
.env.*
dist
build
*.log
.pytest_cache
.DS_Store
```

Dockerfile

Copy

```
FROM python:3.11-slim # pinned, slim, not :latest

ENV PYTHONUNBUFFERED=1 \
    LOG_LEVEL=info # unbuffered logs; both overridable with -e

LABEL org.opencontainers.image.source="https://github.com/my-org/myapp"

RUN useradd --create-home --uid 10001 appuser
WORKDIR /app

COPY requirements.txt . # manifest first: cached install
RUN pip install --no-cache-dir -r requirements.txt

COPY --chown=appuser:appuser . . # source last

USER 10001 # non-root
EXPOSE 8000 # documentation only
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

compose.yaml

Copy

```
services:
  api:
    build: .
    ports: ['8000:8000']
    env_file: [.env]
    depends_on:
      db: { condition: service_healthy } # readiness, not just started
    mem_limit: 512m
    restart: unless-stopped
```



```
POSTGRES_PASSWORD: secret
POSTGRES_DB: app
volumes: ['pgdata:/var/lib/postgresql/data'] # named volume for state
healthcheck:
  test: ['CMD-SHELL', 'pg_isready -U postgres']
  interval: 10s
  retries: 5
  start_period: 30s

volumes:
  pgdata:
```

BASH

Copy

```
docker compose up -d --build
docker compose logs -f api
docker compose exec api sh
docker compose down # add -v only if you mean to delete the database
```

Eight details in that setup carry the lesson of this page: a `.dockerignore` written before the first build, unbuffered logging, the dependency manifest copied before the source, a pinned slim base, a non-root numeric user, an exec-form `CMD` binding `0.0.0.0`, a named volume for anything that must survive, and a health check so the API waits for a database that accepts connections.

Mid-level tips take each of these further: cache keys and why CI never hits the cache, shrinking an image in order of leverage, Compose override traps, entrypoint scripts that behave, volume backup, registry hygiene, and diagnosing "works locally, not in staging".



PREVIOUS

Beginner · Interview Review

NEXT UP

Mid-level · Detailed





MLOps MENA
COMMUNITY

العربية العربية



Join the Community ↗



MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

Roadmaps

Student Guides

Courses

The MLOps Practitioner

Sessions

Articles

COMMUNITY

Meet the team

Services for companies

FAQ

Contact us

Privacy Policy

WHERE WE ARE

WhatsApp

Discord

LinkedIn

X

YouTube

GitHub



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



Learning space

Free to join this week with code **FREECORETEAM**

© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.